Official Documentation

1. The Core Engine (Classes)

This is the foundational code. You must have this pasted into a file (e.g., `signal_lti.py`) or at the top of your test script.

Python

```
import numpy as np

def readable_time_ticks(time_values, max_labels=18):
    if len(time_values) <= max_labels:
        return time_values
    step = int(np.ceil(len(time_values) / max_labels))
    ticks = time_values[::step]
    if ticks[-1] != time_values[-1]:
        ticks.append(time_values[-1])
    return ticks

class DiscreteSignal:
    """Finite discrete-time signal with integer indices."""
    def __init__(self, start_time, end_time):
        self.start_time = start_time
        self.end_time = end_time
        self.values = np.zeros(end_time - start_time + 1, dtype=float)

    def __len__(self):
        return self.end_time - self.start_time + 1

    def times(self):
        return range(self.start_time, self.end_time + 1)

    def get_value_at_time(self, t):
        if self.start_time <= t <= self.end_time:
            return self.values[t - self.start_time]
        return 0.0

    def set_value_at_time(self, t, value):
        if self.start_time <= t <= self.end_time:
            self.values[t - self.start_time] = float(value)
        else:
            raise ValueError("Time is outside the stored range.")
```

```
    def shift(self, k):
        shifted = DiscreteSignal(self.start_time + k, self.end_time + k)
        shifted.values = self.values.copy()
        return shifted

    def add(self, other):
        nstart = min(self.start_time, other.start_time)
        nend = max(self.end_time, other.end_time)
        res = DiscreteSignal(nstart, nend)
        for t in res.times():
            val = self.get_value_at_time(t) + other.get_value_at_time(t)
            res.set_value_at_time(t, val)
        return res

    def multiply(self, scalar):
        res = DiscreteSignal(self.start_time, self.end_time)
        res.values = self.values * scalar
        return res

    def nonzero_samples(self, tolerance=1e-12):
        indices = np.where(np.abs(self.values) > tolerance)[0]
        return [(self.start_time + i, self.values[i]) for i in indices]

    def plot(self, title, save_path=None, ax=None):
        import matplotlib.pyplot as plt
        if ax is None:
            _, ax = plt.subplots()
        time_values = list(self.times())
        markerline, stemlines, baseline = ax.stem(time_values, self.values)
        markerline.set_markersize(6)
        baseline.set_color("black")
        baseline.set_linewidth(1)
        ax.axhline(0, color="black", linewidth=0.8)
        ax.set_title(title)
        ax.set_xlabel("n")
        ax.set_ylabel("value")
        ax.grid(True, alpha=0.35)
        ax.set_xticks(readable_time_ticks(time_values))
        ax.tick_params(axis="x", labelsize=9)
        if save_path is not None:
            plt.savefig(save_path, bbox_inches="tight", dpi=150)
        return ax

class LTISystem:
    """Discrete-time LTI system defined by impulse response."""
```

```
    def __init__(self, impulse_response):
        self.impulse_response = impulse_response

    def output_range(self, input_signal):
        start = input_signal.start_time + self.impulse_response.start_time
        end = input_signal.end_time + self.impulse_response.end_time
        return start, end

    def get_response_components(self, input_signal):
        components = []
        for k, x_k in input_signal.nonzero_samples():
            hshift = self.impulse_response.shift(k)
            hultimate = hshift.multiply(x_k)
            components.append((k, hultimate))
        return components

    def output_by_superposition(self, input_signal):
        start, end = self.output_range(input_signal)
        y = DiscreteSignal(start, end)
        for k, comp in self.get_response_components(input_signal):
            y = y.add(comp)
        return y

    def get_contributions_at_time(self, input_signal, n):
        cont = []
        for k, x_k in input_signal.nonzero_samples():
            hval = self.impulse_response.get_value_at_time(n - k)
            if abs(hval) > 1e-12:
                cont.append((k, x_k, hval, x_k * hval))
        return cont

    def output_at_time(self, input_signal, n):
        total_sum = 0.0
        contributions = self.get_contributions_at_time(input_signal, n)
        for k, x_k, hval, prod in contributions:
            total_sum += prod
        return total_sum

    def output(self, input_signal):
        start, end = self.output_range(input_signal)
        y = DiscreteSignal(start, end)
        for n in y.times():
            y.set_value_at_time(n, self.output_at_time(input_signal, n))
        return y

    def output_super(self, supersignal):
```

```
        """Processes a Supersignal object via the superposition principle."""
        if not supersignal.signals:
            return DiscreteSignal(0, 0)
        total_output = self.output(supersignal.signals[0])
        for i in range(1, len(supersignal.signals)):
            current_output = self.output(supersignal.signals[i])
            total_output = total_output.add(current_output)
        return total_output

class Supersignal:
    """Container for multiple DiscreteSignal instances."""
    def __init__(self, signals):
        self.signals = signals
```

2. Universal Parsing & Helper Functions

Use these snippets to quickly read files or verify outputs.

Reading Standard Input (Comma or Space Separated)

Python

```
def get_array_from_input(prompt_text):
    raw = input(prompt_text)
    # Replaces commas with spaces to handle both formats safely
    return [float(x) for x in raw.replace(',', ' ').split()]
```

Reading 2-Line Text Files (Start/End on Line 1, Values on Line 2)

Python

```
def read_lab_signal(file_path):
    with open(file_path, "r", encoding="utf-8") as f:
        lines = f.read().strip().splitlines()
    start_time, end_time = map(int, lines[0].split())
    values = list(map(float, lines[1].split()))

    sig = DiscreteSignal(start_time, end_time)
    sig.values[:] = values
    return sig
```

Verifying Two Signals

Python

```
def max_absolute_difference(sig1, sig2):
    min_time = min(sig1.start_time, sig2.start_time)
    max_time = max(sig1.end_time, sig2.end_time)
    max_diff = 0.0
    for n in range(min_time, max_time + 1):
        diff = abs(sig1.get_value_at_time(n) - sig2.get_value_at_time(n))
        if diff > max_diff:
            max_diff = diff
    return max_diff
```

3. Scenario Templates (The "Onlines")

Scenario A: Moving Averages (Simple & Weighted)

Core Concept: Create a weight window array $h[n]$ that sums to 1.0. Slide it across the data $x[n]$. Extract only the valid overlapping items.

Python

```
m = len(prices) # Total stock prices
x = DiscreteSignal(0, m - 1)
x.values[:] = prices

# Unweighted: Every day has weight = 1/window_size
h_unweighted = DiscreteSignal(0, window_size - 1)
h_unweighted.values[:] = [1.0 / window_size] * window_size

# Weighted: Most recent day gets highest weight
h_weighted = DiscreteSignal(0, window_size - 1)
sum_weights = (window_size * (window_size + 1)) / 2.0
for k in range(window_size):
    h_weighted.set_value_at_time(k, (window_size - k) / sum_weights)

# Convolve
sys = LTISystem(h_weighted)
y = sys.output(x)

# Extract only the valid data points
valid_outputs = [y.get_value_at_time(n) for n in range(window_size - 1, m)]
```

Scenario B: Exponential Smoothing

Core Concept: Same as moving averages, but the weights follow an exponential decay formula.

Python

```
h = DiscreteSignal(0, window_size - 1)
for k in range(window_size):
    weight = alpha * ((1.0 - alpha) ** k)
    h.set_value_at_time(k, weight)
```

Scenario C: Polynomial Multiplication

Core Concept: The coefficients of a polynomial map perfectly to signal amplitudes. Degree $d \rightarrow$ signal from 0 to d. Multiplication is literally just convolution without any cropping at the end.

Python

```
x = DiscreteSignal(0, degree_1)
x.values[:] = coeffs_1

h = DiscreteSignal(0, degree_2)
h.values[:] = coeffs_2

sys = LTISystem(h)
y = sys.output(x)

final_degree = degree_1 + degree_2
final_coeffs = [int(round(y.get_value_at_time(n))) for n in y.times()]
```

Scenario D: First Difference & Signal Arithmetic

Core Concept: Compute discrete derivatives $x[n] - x[n-1]$ using only the `DiscreteSignal` operations.

Python

```
# To do: signal - shifted_signal
x_delayed = x.shift(1)
negative_x_delayed = x_delayed.multiply(-1.0)
delta_x = x.add(negative_x_delayed)
```

Scenario E: Block Diagrams (Series & Parallel)

Core Concept:

- **Parallel:** Add the impulse responses together.

- **Series (Cascaded):** Convolve them together (treat one as the input to the other).

Python

```
# Parallel Branches
h_parallel = h1.add(h2)

# Series Branches (h_parallel feeds into h3)
sys_h3 = LTISystem(h3)
h_combined = sys_h3.output(h_parallel) # This yields the single master impulse
response
```

4. Cheat Sheet / API Reference

Action / Goal	Code Strategy	What to watch out for
Instantiate a signal	`sig = DiscreteSignal(start, end)`	Start must be $\leq$ End.
Fill a signal	`sig.values[:] = [list_of_floats]`	The list length MUST exactly equal the signal length.
Shift a signal	`new_sig = sig.shift(k)`	Positive k shifts right (delay). Negative k shifts left.
Scale a signal	`new_sig = sig.multiply(scalar)`	Useful for subtracting signals (multiply by `-1` then `add()`).
Add two signals	`sum_sig = sig1.add(sig2)`	Automatically calculates the new expanded time bounds.
Process an LTI System	`sys = LTISystem(h)` `y = sys.output(x)`	The result y will span from $x_{start} + h_{start}$ to $x_{end} + h_{end}$.
Superposition Proof	`sys.output_super(Supersignal([x1, x2]))`	Should equal `sys.output(x1.add(x2))`.
Create a Delta/Impulse	`h = DiscreteSignal(0, 0)` `h.values[:] = [1.0]`	Convolving with this produces an exact copy of the input.

5. Potential Curveball Questions to Prepare For

1. **Correlation instead of Convolution:** If they ask you to perform Cross-Correlation ($x[n] \star h[n]$), remember that it is just convolution where $h[n]$ is **time-reversed**. You may need to write a quick helper function to flip a signal (swap start/end times and reverse the `values` array).
2. **Echo / Reverberation Systems:** You might be asked to model a room echo. The impulse response will look like an impulse at $n = 0$, followed by smaller impulses delayed in time (e.g., $h[n] = \delta[n] + 0.5\delta[n - 100] + 0.25\delta[n - 200]$). Use a dictionary and `signal.set_value_at_time()` to build sparse signals like this.
3. **Cascading Multiple Filters:** They might ask you to apply a moving average filter, and *then* apply a first-difference filter to the result to find trends. You just cascade the systems: $y_{temp} = sys_1.output(x)$, followed by $y_{final} = sys_2.output(y_{temp})$.

##Deconvolution

The Mathematical Intuition

In discrete-time convolution, the very first value of your output $y[0]$ is simply the product of the first input value and the first impulse response value:

$$y[0] = x[0] \times h[0]$$

If you want to find $x[0]$, you just rearrange the algebra:

$$x[0] = \frac{y[0]}{h[0]}$$

For the next time step, $y[1]$ is created by the overlap of two terms:

$$y[1] = (x[1] \times h[0]) + (x[0] \times h[1])$$

Since you already figured out $x[0]$ in the previous step, you can plug it in and solve for $x[1]$:

$$x[1] = \frac{y[1] - (x[0] \times h[1])}{h[0]}$$

This creates a recursive pattern. To find any value $x[n]$, you take $y[n]$, subtract all the overlapping effects of the x values you have already calculated, and divide by $h[0]$.

The general formula is:

$$x[n] = \frac{1}{h[0]} \left(y[n] - \sum_{k=0}^{n-1} x[k]h[n-k] \right)$$

The Algorithmic Steps

To write this in code, you follow three strict rules:

1. **Calculate the new bounds:** Because convolution expands a signal's length ($len_y = len_x + len_h - 1$), deconvolution shrinks it back down. The length of the recovered input will be $len_y - len_h + 1$. Its start time will be $start_y - start_h$.
2. **Verify the divisor:** The very first element of your impulse response ($h[0]$) **cannot be zero**. If it is, you cannot divide by it, and the algorithm fails.
3. **Iterate and subtract:** Loop through the calculated length of $x[n]$, applying the recursive formula step-by-step.

The Python Implementation

Here is how you can write a standalone deconvolution function that seamlessly takes your `DiscreteSignal` objects and returns the recovered input.

Python

```
def deconvolute(y_signal, h_signal):
    # 1. Determine the time bounds and length for the recovered x[n]
    x_start = y_signal.start_time - h_signal.start_time
    x_len = len(y_signal) - len(h_signal) + 1
    x_end = x_start + x_len - 1

    # Initialize the blank recovered signal
    x_signal = DiscreteSignal(x_start, x_end)

    # 2. Grab the divisor (the very first element of h[n])
    h_first = h_signal.values[0]
    if h_first == 0.0:
        raise ValueError("Deconvolution failed: The first value of h[n] cannot
be zero.")

    # 3. Apply the recursive subtraction algorithm
    for n in range(x_len):
        current_y = y_signal.values[n]
        overlap_sum = 0.0

        # Calculate the sum of the previously found x values overlapping with
h
        for k in range(n):
            if (n - k) < len(h_signal):
                overlap_sum += x_signal.values[k] * h_signal.values[n - k]

        # Lock in the new x value
        x_signal.values[n] = (current_y - overlap_sum) / h_first
```

```
    return x_signal
```

1. The 2D Signal Class

This handles a 2D NumPy matrix instead of a 1D array. It tracks boundaries for both the X (rows) and Y (columns) axes.

```
import numpy as np

class DiscreteSignal2D:
    """Finite discrete-time 2D signal (e.g., an image or matrix)."""

    def __init__(self, x_start, x_end, y_start, y_end):
        self.x_start = x_start
        self.x_end = x_end
        self.y_start = y_start
        self.y_end = y_end

        # Calculate matrix dimensions based on coordinates
        width = x_end - x_start + 1
        height = y_end - y_start + 1
        self.values = np.zeros((width, height), dtype=float)

    def get_value(self, x, y):
        """Safely returns the value at (x, y) or 0.0 if out of bounds."""
        if (self.x_start <= x <= self.x_end) and (self.y_start <= y <=
self.y_end):
            return self.values[x - self.x_start, y - self.y_start]
        return 0.0

    def set_value(self, x, y, value):
        """Updates the sample value at coordinate (x, y)."""
        if (self.x_start <= x <= self.x_end) and (self.y_start <= y <=
self.y_end):
            self.values[x - self.x_start, y - self.y_start] = float(value)
        else:
            raise ValueError(f"Coordinates ({x}, {y}) are outside the stored
range.")

    def nonzero_samples(self, tolerance=1e-12):
        """Finds all non-zero elements to optimize the convolution loops."""
        nonzero_indices = np.where(np.abs(self.values) > tolerance)
        samples = []
        # nonzero_indices[0] is X offsets, nonzero_indices[1] is Y offsets
```

```
        for i in range(len(nonzero_indices[0])):
            x_idx = nonzero_indices[0][i]
            y_idx = nonzero_indices[1][i]
            val = self.values[x_idx, y_idx]
            samples.append((self.x_start + x_idx, self.y_start + y_idx, val))
        return samples
```

2. The 2D LTI System Class

This expands the convolution sum to handle 2D indexing:
$y[m, n] = \sum_j \sum_k x[j, k] \times h[m - j, n - k]$

```
class LTISystem2D:
    """2D Discrete-time LTI system defined by a 2D impulse response
(kernel)."""

    def __init__(self, impulse_response_2d):
        self.h = impulse_response_2d

    def output_range(self, input_signal):
        """Calculates the expanded bounding box of the output 2D matrix."""
        out_x_start = input_signal.x_start + self.h.x_start
        out_x_end = input_signal.x_end + self.h.x_end
        out_y_start = input_signal.y_start + self.h.y_start
        out_y_end = input_signal.y_end + self.h.y_end

        return out_x_start, out_x_end, out_y_start, out_y_end

    def output_at_coords(self, input_signal, m, n):
        """Executes the 2D convolution sum for a single output
pixel/coordinate."""
        total_sum = 0.0

        # We only loop through the non-zero inputs to save processing time
        for j, k, x_val in input_signal.nonzero_samples():
            # Check the flipped and shifted impulse response at (m-j, n-k)
            h_val = self.h.get_value(m - j, n - k)
            if abs(h_val) > 1e-12:
                total_sum += x_val * h_val

        return total_sum

    def output(self, input_signal):
        """Generates the full 2D convoluted output signal."""
```

```
        x_start, x_end, y_start, y_end = self.output_range(input_signal)
        y_signal = DiscreteSignal2D(x_start, x_end, y_start, y_end)

        # Slide the kernel across the entire output coordinate space
        for m in range(x_start, x_end + 1):
            for n in range(y_start, y_end + 1):
                val = self.output_at_coords(input_signal, m, n)
                y_signal.set_value(m, n, val)

        return y_signal
```

3. How to Test It (Example Usage)

If you want to prove to a TA that this works perfectly, you can run a classic "Edge Detection" convolution on a tiny 3x3 pixel matrix.

```
if __name__ == "__main__":
    # 1. Create a 3x3 input image (e.g., coordinates 0 to 2 for both X and Y)
    image = DiscreteSignal2D(0, 2, 0, 2)
    # A simple vertical line of high values in the middle
    image.values[:] = np.array([
        [0, 10, 0],
        [0, 10, 0],
        [0, 10, 0]
    ])

    # 2. Create a 2x2 edge-detection kernel (Sobel-style)
    kernel = DiscreteSignal2D(0, 1, 0, 1)
    kernel.values[:] = np.array([
        [-1, 1],
        [-1, 1]
    ])

    # 3. Apply the 2D LTI System
    sys = LTISystem2D(kernel)
    output_image = sys.output(image)

    # 4. Show results
    print("--- 2D Convolution Output Matrix ---")
    print(output_image.values)
```